

# How we run work here

Everything you need to use the ops platform — whether you write code or not. Read it once, keep it open for a week.

[ihelp-ops-ai-decodeds-projects.vercel.app](https://ihelp-ops-ai-decodeds-projects.vercel.app)

---

## Nobody types how far along they are.

There is no “80% done” box anywhere in this platform, and there never will be. A task moves because something real happened: a branch appeared, a commit landed, a pull request opened, a person approved it, it merged.

**You can't fall behind by forgetting to update a status.** If you did the work, the platform already knows. You never have to report progress.

**You also can't get ahead by saying you did.** Nobody can, including the founder. That's the point — everyone is measured the same way, by what exists.

## Getting in

Two minutes, once.

Open the link above and press **Sign in with Google**, using your work Gmail. Your account is created the first time you sign in. That's the whole setup.

## If you write code, link GitHub too

Ask whoever runs the platform to attach your GitHub username to your account — it's a one-line change.

Git signs every commit with a GitHub username. Reviews, approvals and code ownership all use it. So an account with no GitHub username attached can't *own* work, and the platform won't let it start a task or run an agent. That's deliberate.

Without GitHub linked you can still read everything, book leave, comment on tasks, see your own goals and feedback — and, if you are a lead, assign work to other people. Only *doing* the work is closed to you: taking a task, starting one, and running an agent.

## The screens

|           |                                                                            |
|-----------|----------------------------------------------------------------------------|
| Board     | Every open task across every repository, and how far each has really got   |
| Team      | Who's on which pod, what they have open, who's away today                  |
| Leave     | Book time off, see your balance, approve your team's requests              |
| Me        | Your goals, your 1:1 notes, feedback about you, what you've shipped        |
| Analytics | How work is flowing across the whole team                                  |
| Admin     | Accounts, roles and agent tiers. Only the CTO and the founder see this one |

The board covers **every repository the company has**, including ones created after you read this — it reads the list from GitHub rather than from a setting somebody has to remember to update. Tasks are grouped under the

repository they belong to, and a task's address carries it, like `/task/ihelprobotics/ihelp-ops/42` . Issue numbers restart in every repository, so “#1” on its own does not say which piece of work you mean.

Repositories with nothing open are not listed — they are counted on one line at the bottom instead. Thirteen headings with “nothing here” under eleven of them buries the two tasks that matter.

## Roles

Four of them. Your role decides what you may do to *other people's* work and records — never what you may do to your own.

|                |                                                                                                                      |
|----------------|----------------------------------------------------------------------------------------------------------------------|
| <b>member</b>  | Take work, do it, run agents up to your tier, book your own leave. Where everybody starts                            |
| <b>lead</b>    | All of that, plus assign work to people, approve their leave, and see their leave and nudges                         |
| <b>cto</b>     | All of that for everybody — and the only role that can read a 1:1 it did not write                                   |
| <b>founder</b> | All of a lead's reach across everybody, and any agent. Deliberately <i>not</i> able to read other people's 1:1 notes |

Separately, an **agent tier** — `week1` , `week2` or `full` — decides which agents you can dispatch. It grows as you learn what good looks like here. Neither your role nor your tier is a judgement; both are a starting point.

**The CTO reading a 1:1 is logged, and you see the log.** It appears on your own page under “who has read notes about you”. That is what makes “the CTO can read everything” something you can live with rather than something you have to trust.

## Who changes a role

The CTO and the founder, on the **Admin** screen — which only they can see. Nobody can change their own row, including them: a role change should be something a second person agreed to. The full table of what every role can do is on that screen, and each line names where it is enforced.

If someone new joins, they sign in with Google and appear there as a `member` on `week1` with no GitHub login. Setting that login is the thing that turns an account into somebody who can own work.

**Every role change is recorded** — who made it, when, and what it was before. The changes to your own account appear on your own page, and nobody can edit or remove a line, including an admin. Your role changing is a thing that happened to you, so it is a thing you can read.

## Getting a task

Every task has **exactly one owner, or nobody**. Never two.

A task with two owners has none. “Somebody was going to do it” is the outcome this whole system exists to prevent, so the platform will not let two names sit on one piece of work.

There are two ways your name gets on one, and both are normal.

### You take it

Find something on the board with *nobody has taken this* under it and press **Take it**. That is the whole thing — no permission, no waiting. If you can see the work, you can start it. The task’s own page has the same control, under **Owner**.

**You cannot take a task somebody else holds.** The button is not offered, and the platform names who has it. Go and ask them — they may be stuck, or finished, or glad to hand it over.

## Somebody gives it to you

Leads, the CTO and the founder can assign anyone. On the task page they get **Assign to...**, or **Reassign** to move a task off whoever holds it now.

That last part earns its place: work can be moved off someone who is stuck, on long leave, or has left. Without it, a task would belong to whoever touched it first for ever.

**You do not need a GitHub account to assign work.** The platform assigns using its own credentials, so a delivery manager who has never written a line of code can hand work out all day. Only *doing* the work — branches, commits, pull requests — needs a linked account, because those get signed with a git identity.

## Giving one back

If it is yours and you have not started, press **Hand it back**. It returns to the board with nobody's name on it, and no explanation is owed. An honest hand-back beats a task sitting under your name for three weeks.

## How a task actually moves

Seven steps. Each is caused by a real thing — never by clicking a button that says “done”.

---

**10%** Somebody opened the issue — it may not have an owner yet

---

**20%** A branch was created for it

---

**40%** The first commit landed on that branch

---

**60%** A pull request was opened

---

**75%** The automated checks passed

90% A human approved it

100% It was merged

If a task sits at 10%, nothing has happened yet — not “somebody forgot to update it”. That’s useful information rather than a failure of admin.

## Doing a piece of work

### 1 Make sure it’s yours

Take it from the board, or check that **Owner** on the task page says your name. Everything below assumes the task is yours — that is what makes the work attributable to you rather than to nobody.

### 2 Start it

Press **Start task**. The platform creates a branch named after the issue — `task/42/fall-detection-threshold`. That number is how everything you do afterwards attaches itself to the task automatically.

### 3 Open it in VS Code

The task page gives you three ways in: **Desktop** opens VS Code on your machine, **Browser** opens an editor in a tab with nothing to install, and **Terminal** gives you the git command if you already have the repo.

### 4 Set up the commit hook — once per clone

```
git config core.hooksPath .githooks
```

Run this the first time you clone the repository. It makes every commit tag itself with the task number, which is what puts your work on the board without you doing anything else. Skip it and your commits still land — they just won’t attach to any task, and the bar won’t move.

### 5 Work, commit, push

Normal git. Commit and push as often as you like. Within a minute of pushing, the task moves to 40% on its own.

6

### **Pull request, review, merge**

All three from the task page. The **Merge** button calls GitHub — it can't override anything. If GitHub refuses because a review is missing, that refusal is the protection working, not a broken button.

## **Using an agent**

Agents are AI teammates. Pick a task, press **Run**, and a few minutes later there's a pull request waiting for you.

### **What you can run depends on your tier**

WEEK 1

Scribe (docs) · Data Annotator · QA

WEEK 2

the above, plus Frontend

FULL

the above, plus Full-stack · AI Developer · Integrator · Architect

Locked agents stay visible and greyed out with the reason showing — a hidden button teaches you nothing. A few (Deployer, Cloud, Social, Outreach, Lead Gen) can never be dispatched from here; their human owner runs them.

**You own what the agent produced.** Your name goes on the pull request. Read every line, run it, and be ready to explain it. “The agent wrote it” isn't an answer, and the platform records who asked for the run.

Two runs at a time, each — the same work-in-progress limit as everything else. Every run records what it cost, visible on the task and on Analytics.

## Booking leave

Go to **Leave**, pick the kind and the dates, press **Request**.

|                |                               |
|----------------|-------------------------------|
| <b>Planned</b> | Comes off your annual balance |
|----------------|-------------------------------|

|               |                               |
|---------------|-------------------------------|
| <b>Unpaid</b> | Comes off your annual balance |
|---------------|-------------------------------|

|             |                                   |
|-------------|-----------------------------------|
| <b>Sick</b> | Recorded, but <b>not</b> deducted |
|-------------|-----------------------------------|

|                 |                                   |
|-----------------|-----------------------------------|
| <b>Comp off</b> | Recorded, but <b>not</b> deducted |
|-----------------|-----------------------------------|

Your lead approves it — or the CTO, if you have no lead set. The page tells you who it's waiting on, and you can withdraw a request while it's still pending. A rejection always comes with a written reason you can read.

**Why leave is in an engineering tool at all:** so nobody chases you on a day you're away. Once it's approved, the daily email skips you entirely. That is the whole reason it exists.

## Your page, and who can read it

**Me** shows your goals, your 1:1 notes, feedback written about you, and a log of who has read notes about you.

Goals are reported as *counts of evidence linked and landed* — never as a score. You will never see a percentage next to your name in this platform.

### Who can read what

This is enforced by the database itself, not by a policy someone could forget.

- **You** can read everything written about you. There are no private manager files. If it exists, you can see it.
- **Whoever wrote a note** can read their own note.



- **The CTO** can read all notes — and every read of a note they didn't write and isn't about them appears in your access log.
- **Nobody else.** Not your lead, not another lead, not a colleague.

If a colleague opens your page and sees nothing under 1:1s, the page tells them *why* it's empty rather than implying you have no record.

## Analytics

Team-wide, never a leaderboard.

|                            |                                                                                                                     |
|----------------------------|---------------------------------------------------------------------------------------------------------------------|
| <b>Cycle time</b>          | How long a task took from opened to merged                                                                          |
| <b>Review wait</b>         | How long a pull request sat before a human approved it. If this climbs, reviews are the bottleneck — not the coding |
| <b>Claimed, and proven</b> | Open tasks with a name on them and no branch, commit or pull request behind them yet                                |
| <b>Stalled</b>             | Nothing has happened for three days or more                                                                         |
| <b>Cost</b>                | What the agents spent, per agent and per person                                                                     |

*Claimed, and proven* is not a naughty list. A task can be assigned this morning, or be a week of reading first. It's simply the one place where what's claimed and what exists sit side by side. Same for stalled — a prompt to ask, not a verdict.

**There are no hours anywhere.** No time online, no time tracked, no presence. The moment those sit next to real work, people optimise the easier number. Cycle time measures how long the *work* waited — not how long you sat at a desk.

## The daily email

At 6pm, two things go out.

A **nudge** — only to someone with nothing recorded that day and no blocker raised. It's a prompt, not a telling-off; plenty of days are reading, debugging or meetings. At most one a day, ever. On approved leave, none at all.

A **digest** to the CTO and delivery manager, covering everybody in one email.

### If you're stuck

Raise it in writing — label the issue `blocked` , or say so in your pod channel.

*A block that is raised belongs to the company. A block that is not raised belongs to you.*

Nobody minds a blocker. Silence is the only thing that causes problems.

## Looks like a bug, isn't

Every error here tells you what failed and what to check. If you find one that doesn't, that's worth reporting.

|                                  |                                                                                                                          |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <b>Everything says 10%</b>       | No GitHub activity has reached the platform. The board says so explicitly rather than showing you a quiet week           |
| <b>Merge was refused</b>         | Branch protection. A required review or check is missing.<br>Working as intended                                         |
| <b>My commits aren't showing</b> | The commit hook probably isn't installed in that clone. Run the <code>core.hooksPath</code> command above and push again |
| <b>I can't run any agents</b>    | Your GitHub account isn't linked yet, or the agent is above your tier. The tile says which                               |

**There's no Take it button** Either the task already has an owner — it says who — or your GitHub account isn't linked yet

---

**The same number twice** Different tasks in different repositories. Issue numbers restart in each one, which is why the repository is in every link

## If you don't write code

Most of the platform is yours.

- Read every task across every repository, and see exactly where each stands
- Comment on tasks — your comment goes to the real GitHub issue
- Book and track leave
- See your goals, your 1:1 notes, and feedback about you
- See the team, and who's away

**And if you are a lead, the CTO or the founder, you can assign work** — hand a task to somebody, or move one off a person who is stuck or away. That needs no GitHub account at all, because the platform assigns using its own credentials.

What you can't do is *start* a task or run an agent. Those write commits and pull requests, which get signed with a git identity, and an account without one cannot own them. That's a limit on how git works, not a comment on you.

---

Questions about the platform go to the founder. Questions about a task go on the task.

The full technical documentation lives in the repository under `docs/`.